

CS-334: Machine Learning

Lecture 08: Logistic Regression

(with In-Class Exercise #7)

09/22/2026

Prof. Shengpu Tang

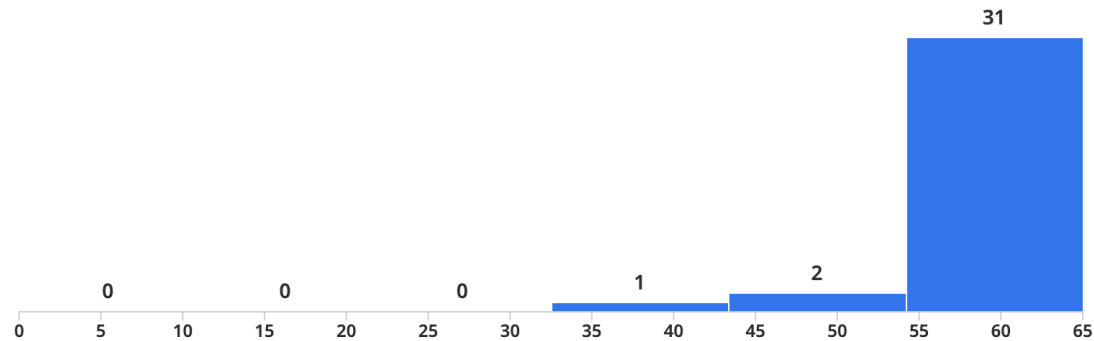


Announcements

- HW1 grade released
- Check canvas for solutions
- Check gradescope for graded homework
- Submit a regrade request if you believe there's an error

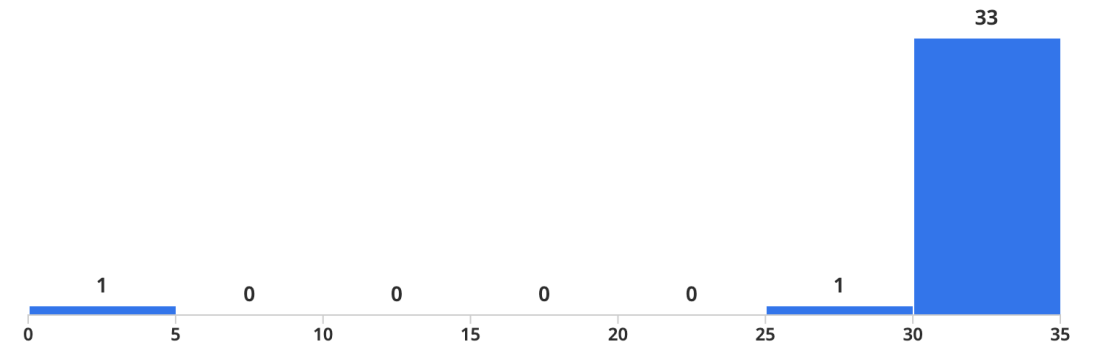
HW1 Grades Released

Review Grades for HW1-Written



Minimum	Median	Maximum	Mean	Std Dev ?
35.0	65.0	65.0	62.47	5.88

Review Grades for HW1-Code



Minimum	Median	Maximum	Mean	Std Dev ?
0.0	34.0	35.0	33.0	5.91

HW2 Reminder

Due Sunday 9/27 11:59pm

Topics covered

- Decision Boundaries (Lec #3)
- Perceptron (Lec #4)
- GD, SGD (Lec #5)
- Linear Regression (Lec #6)
- Regularization (Lec #7)

Extra Credit due Wed 9/30 11:59pm

- Weighted Linear Regression

Submission Instructions: The homework is due on Gradescope in **two** parts.

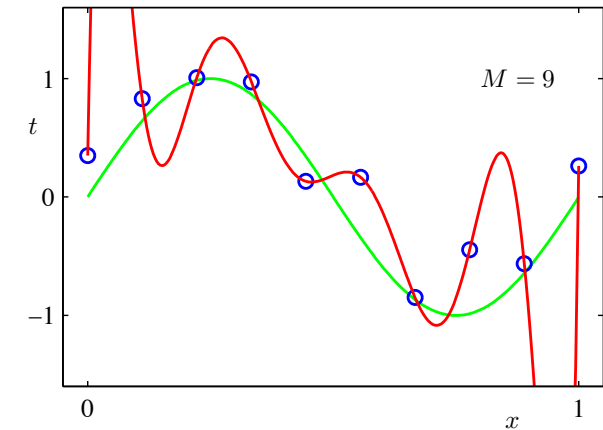
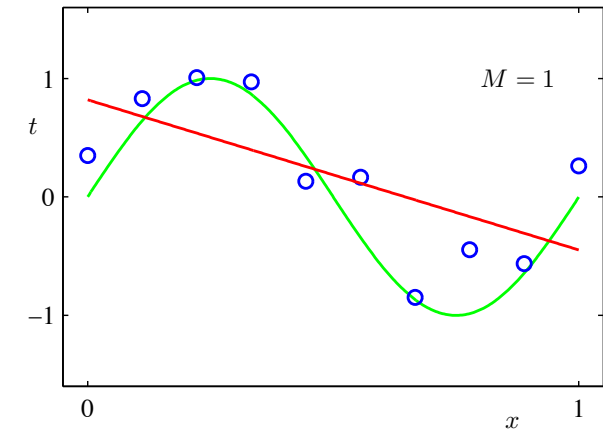
- **Upload PDF to HW2-Written:** Create a single high-quality PDF with your answers to the non-coding problems. Your submission may be typed or handwritten (can be scanned or from a note-taking software), but the pages must be tagged with the relevant questions appropriately on Gradescope. You do not need to include code in the PDF unless otherwise instructed.
- **Submit code to the HW2-Code:** Questions marked with ✂ on the left margin are graded by a Python autograder. Your submission must include only the following files: `perceptron.py`, `linear_regression.py`, `README.txt` (no data files please). You may submit multiple times but be sure to upload *ALL* files when you re-submit; only the latest submission will be considered. The `README.txt` file must contain a **SIGNED** honor code statement that reads as follows:

```
THIS HOMEWORK IS MY OWN WORK, WRITTEN WITHOUT COPYING FROM OTHER STUDENTS  
OR DIRECTLY FROM LARGE LANGUAGE MODELS SUCH AS CHATGPT.  
Any collaboration or external resources have been properly acknowledged.  
<add details>  
/* Your_Name_Here */
```

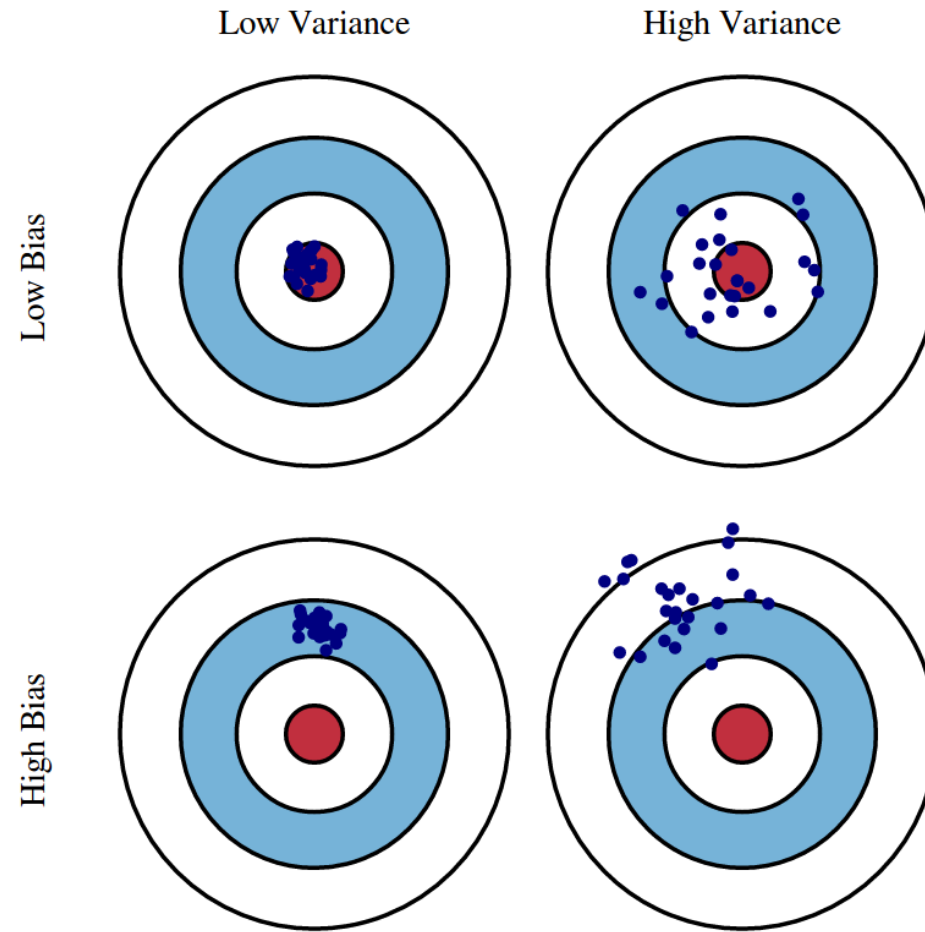
Note about runtime: a correct implementation should take <1 minute for all experiments in this homework.

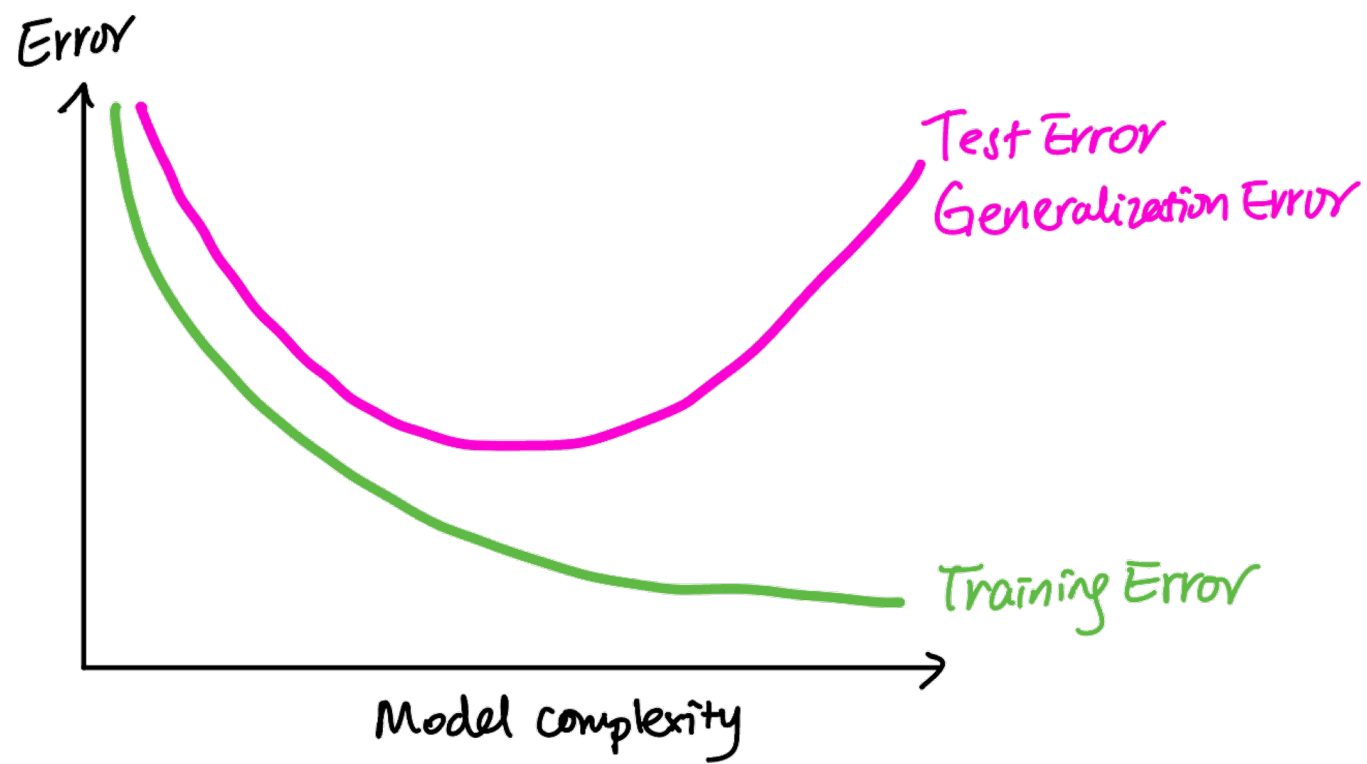
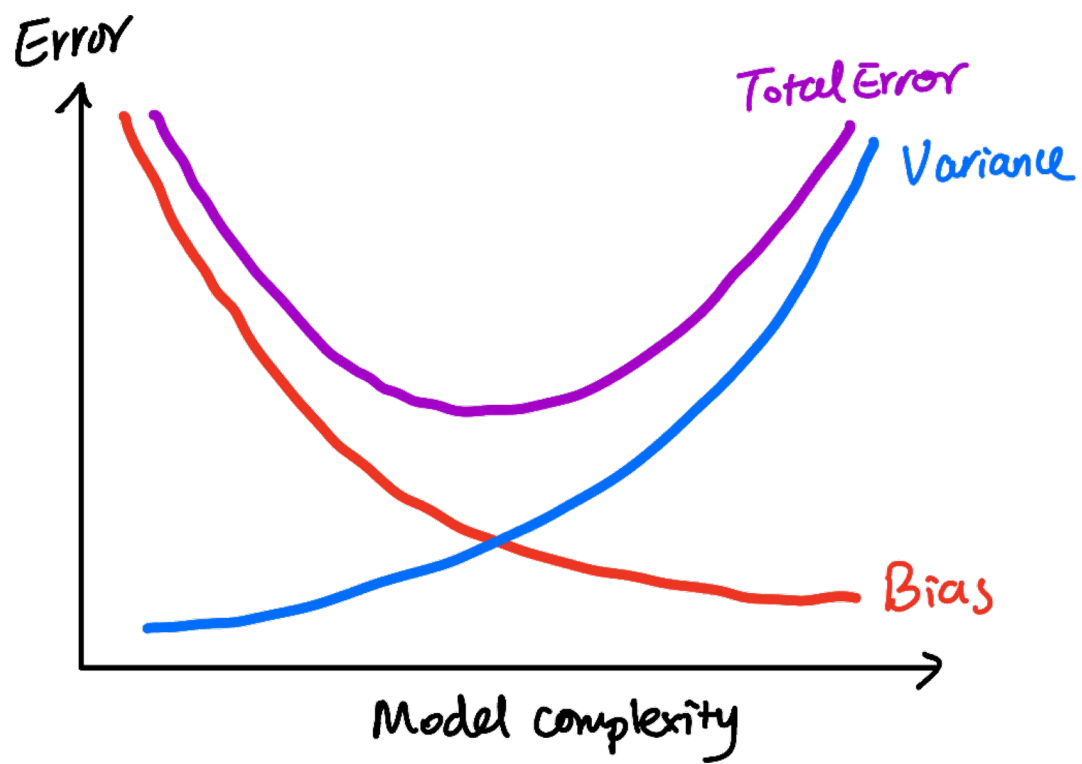
Recap: Bias-Variance Trade-off

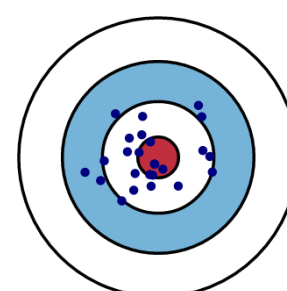
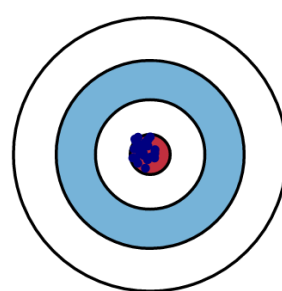
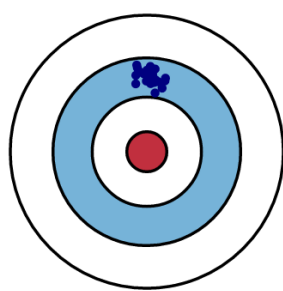
- Model too “simple” $\rightarrow$ does not fit data well
- Model too complex $\rightarrow$ small changes to the data changes the solution a lot



Recap: Bias-Variance Trade-off

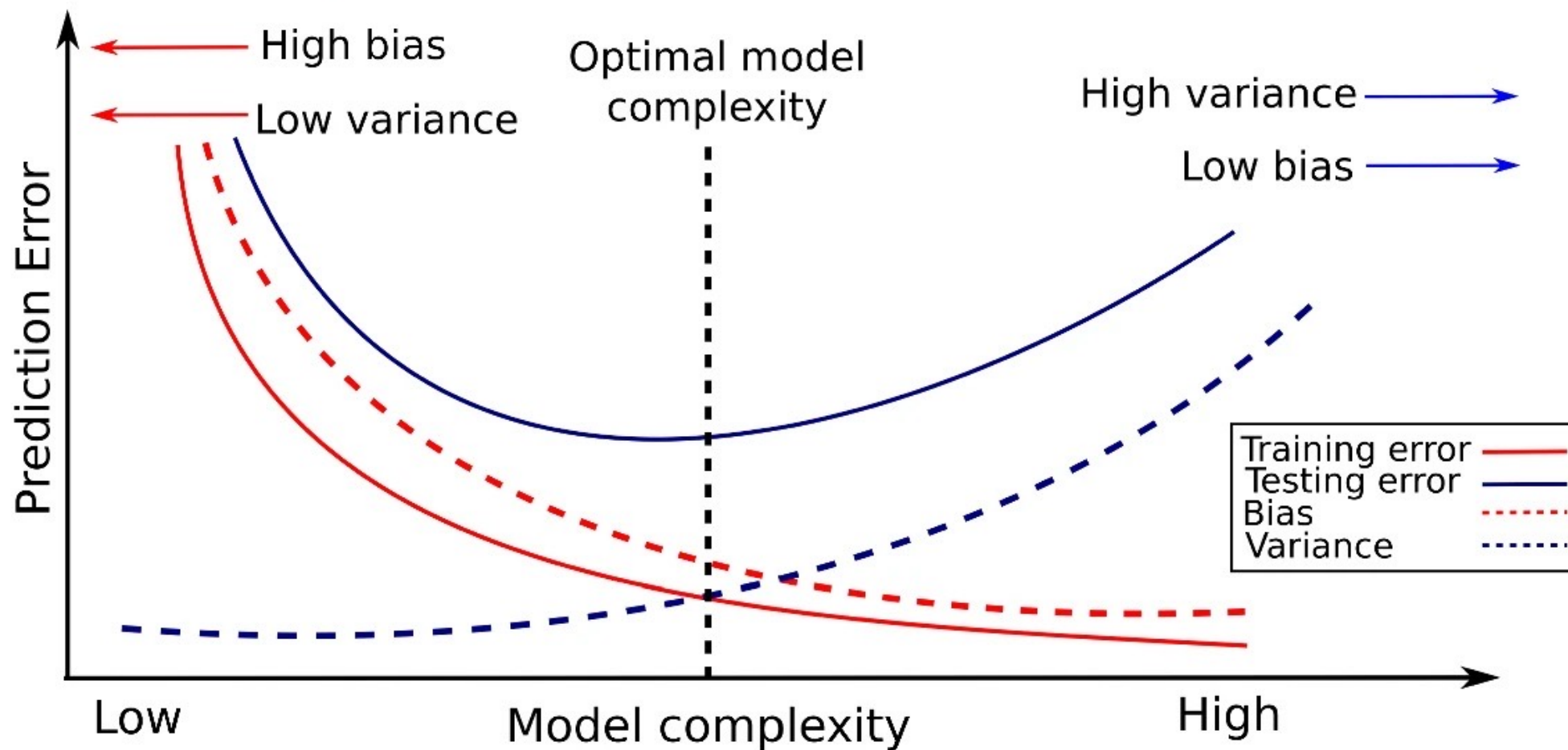






Underfitting

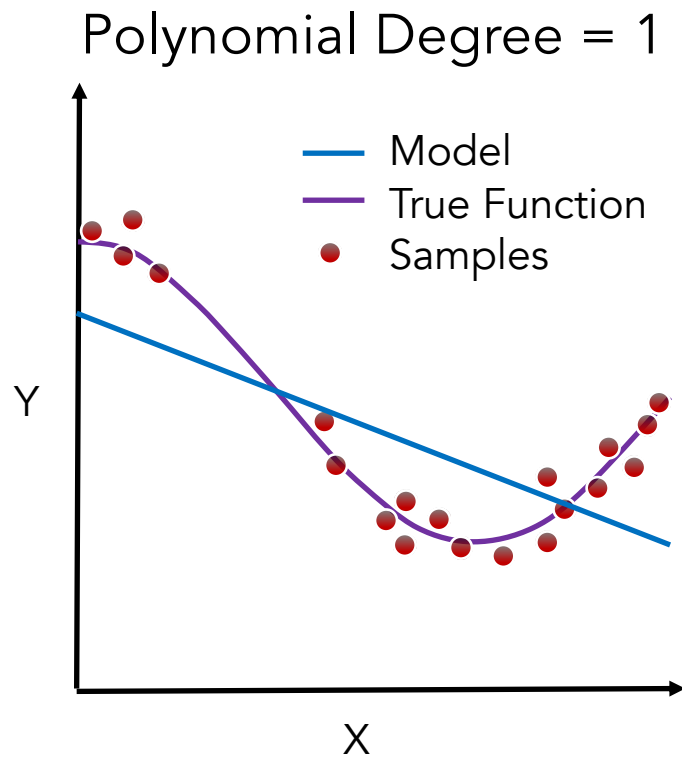
Overfitting



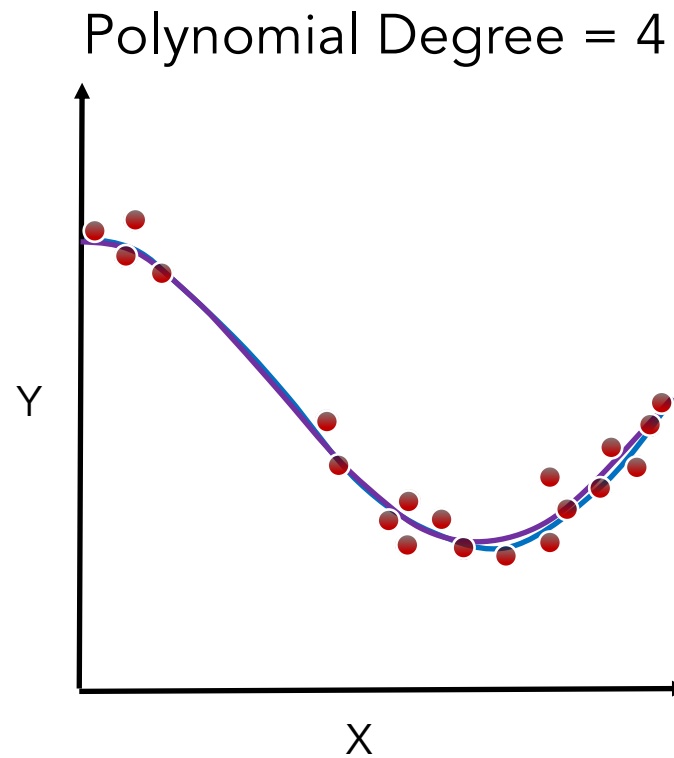
Recap: Bias-Variance Trade-Off

- Generalization — model performance of a model on independent / future unseen data (data not used in training)
- Underfitting – model is unable to capture the relationship between the input and output variables accurately;
high error on both training and test data
- Overfitting — model is specific to the training set and is learning the noise from the data instead of generalizable rule;
low error on training but high error on test data

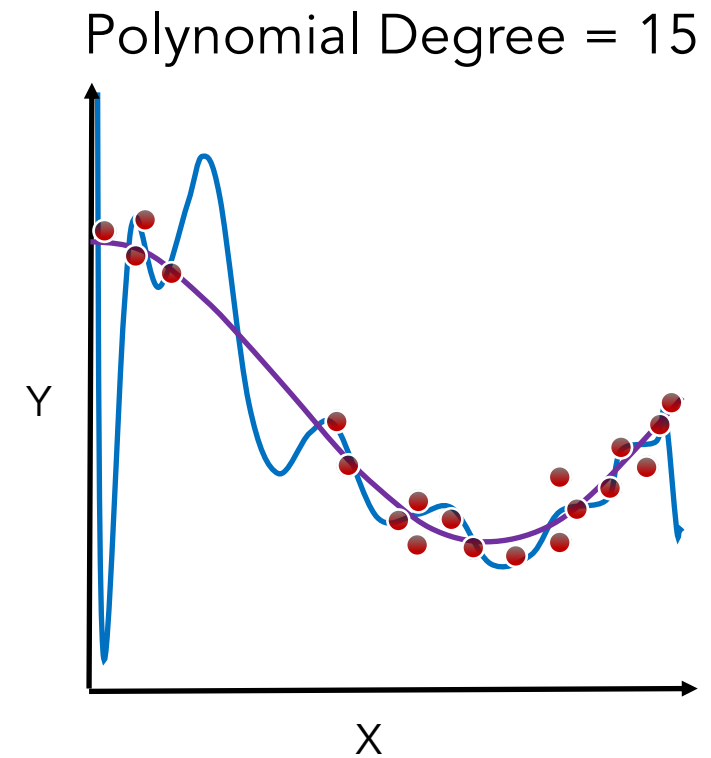
Model Generalization



Poor on Training Set
Poor at Predicting

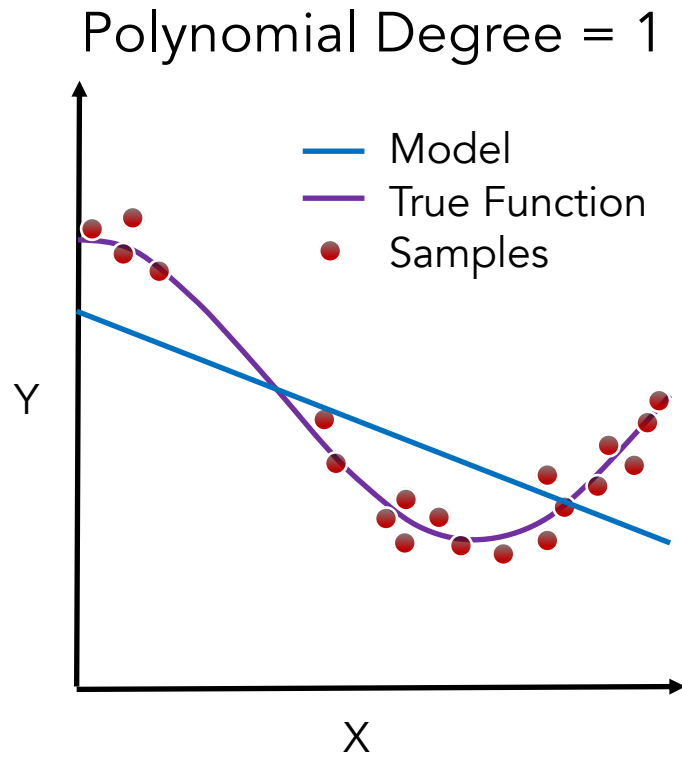


Generalizable

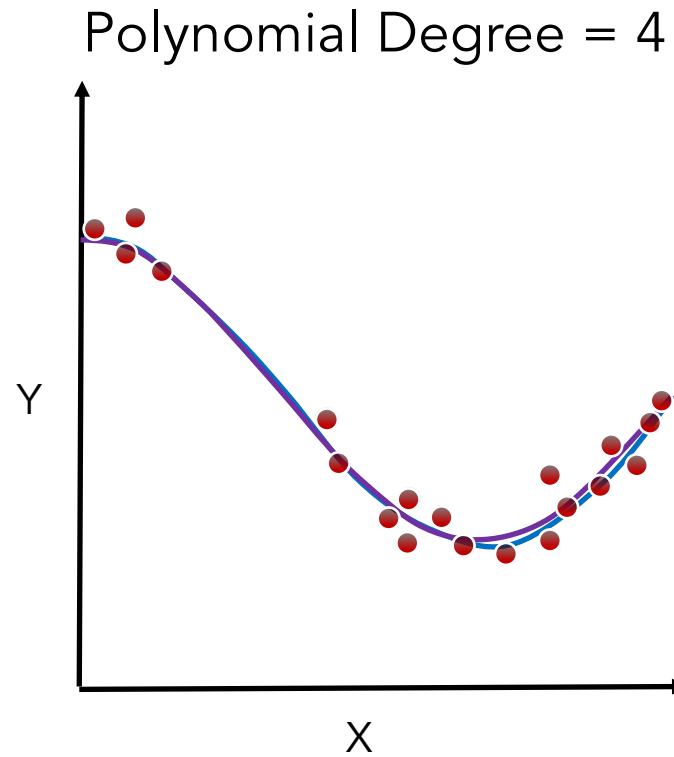


Very Good on Training Set
Poor at Predicting

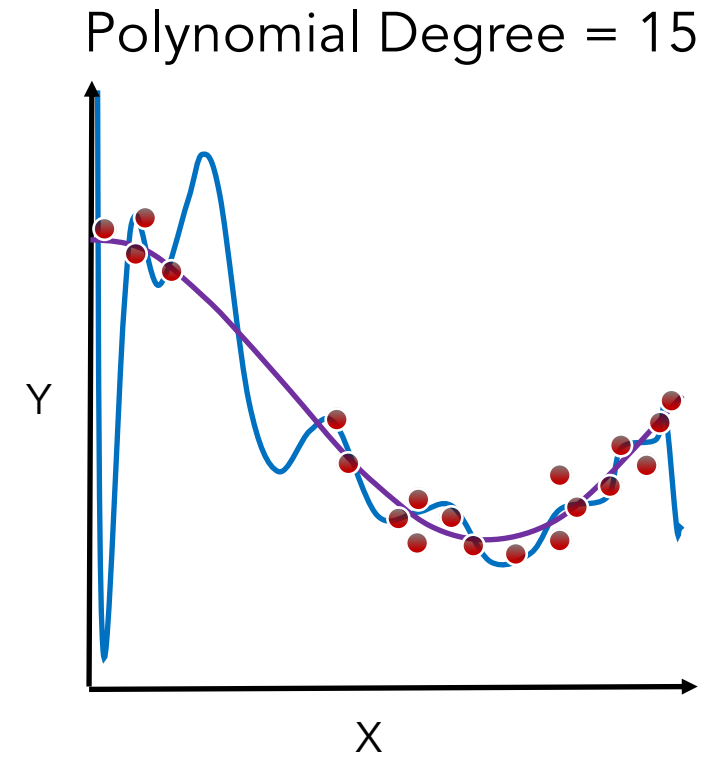
Underfitting vs Overfitting



Underfitting

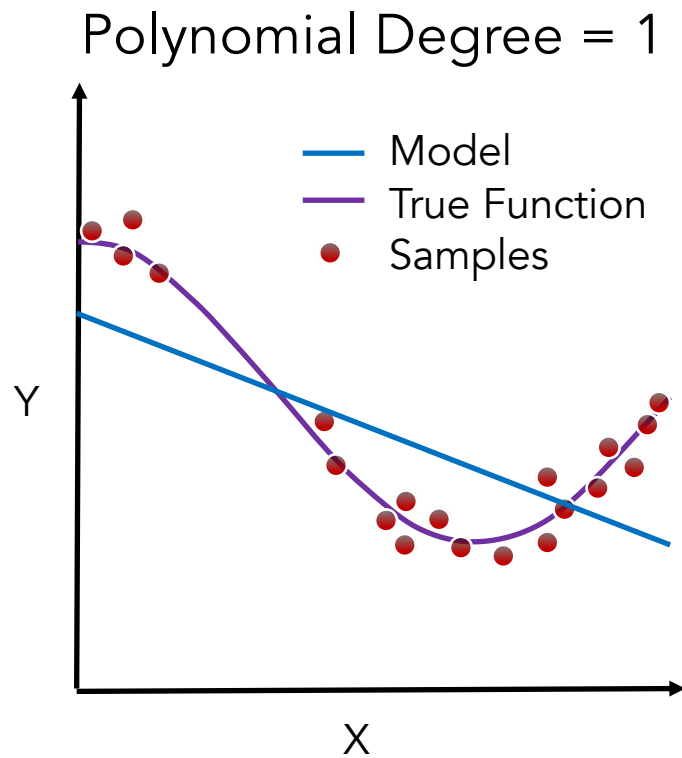


Just Right

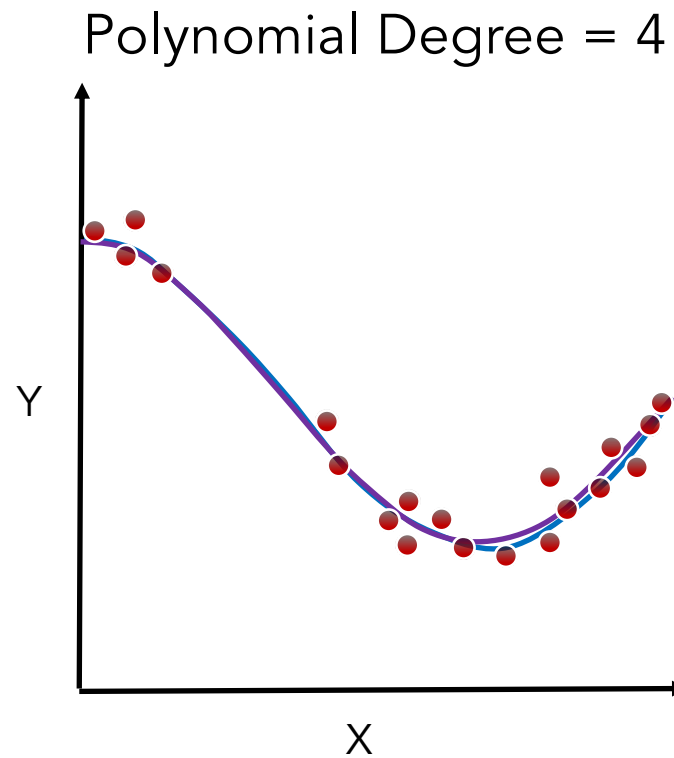


Overfitting

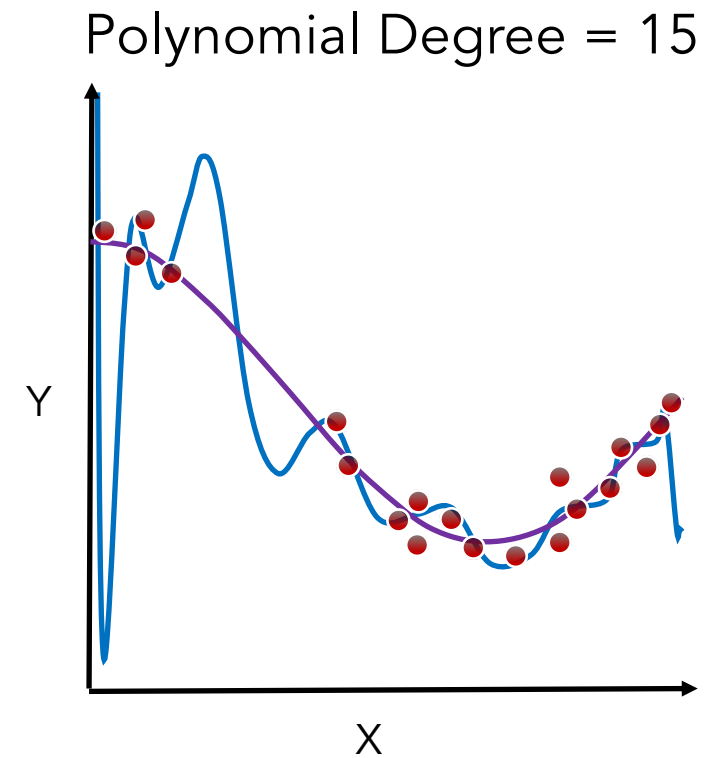
Bias-variance trade-off



High Bias
Low Variance

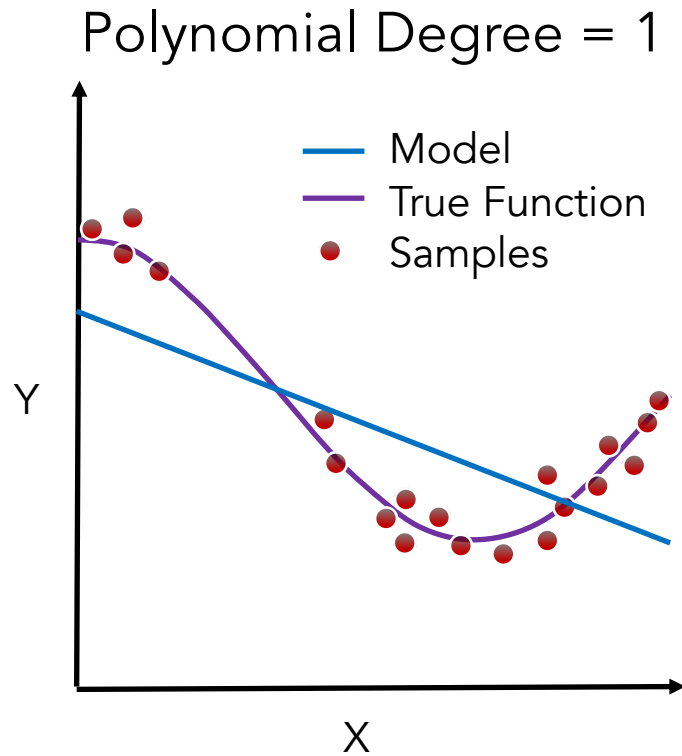


Low Bias
Low Variance

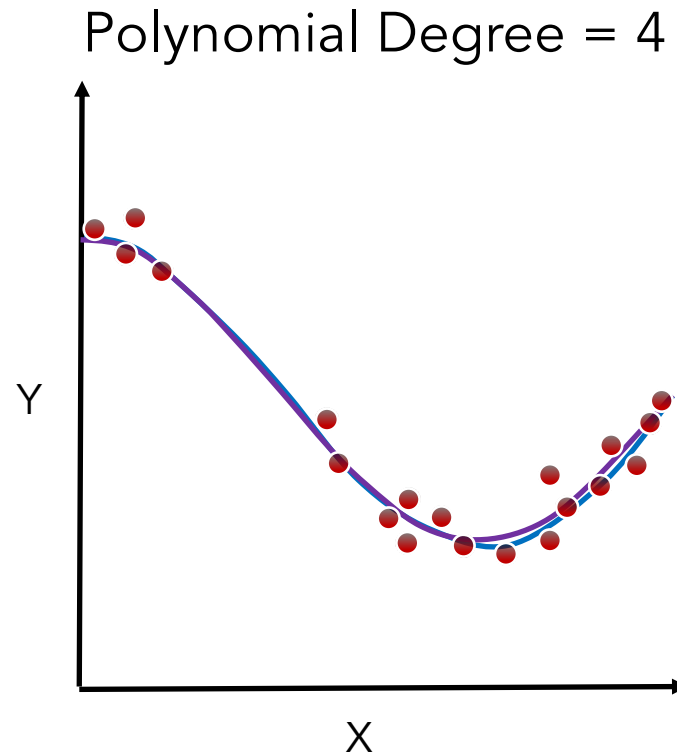


Low Bias
High Variance

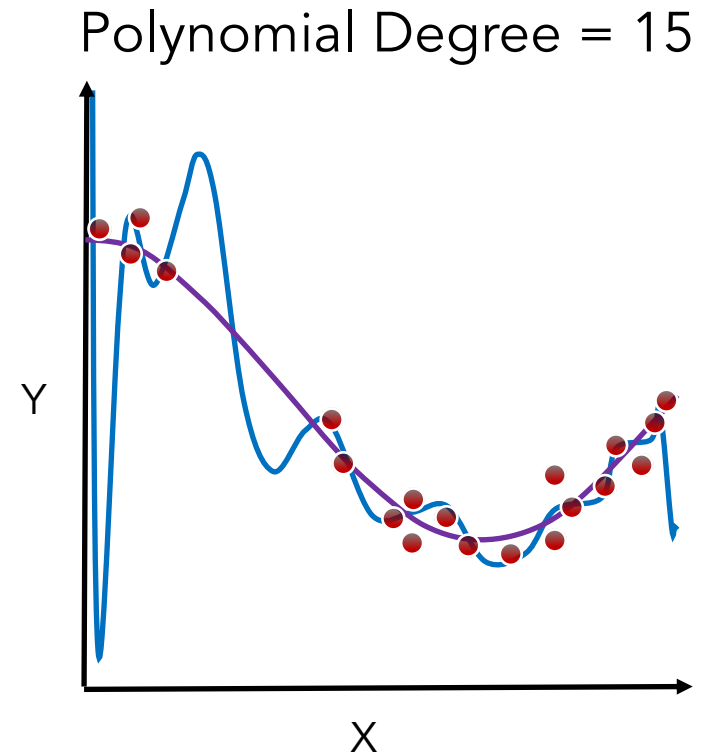
Approximation Error vs Estimation Error



High Approximation Error
Low Estimation Error

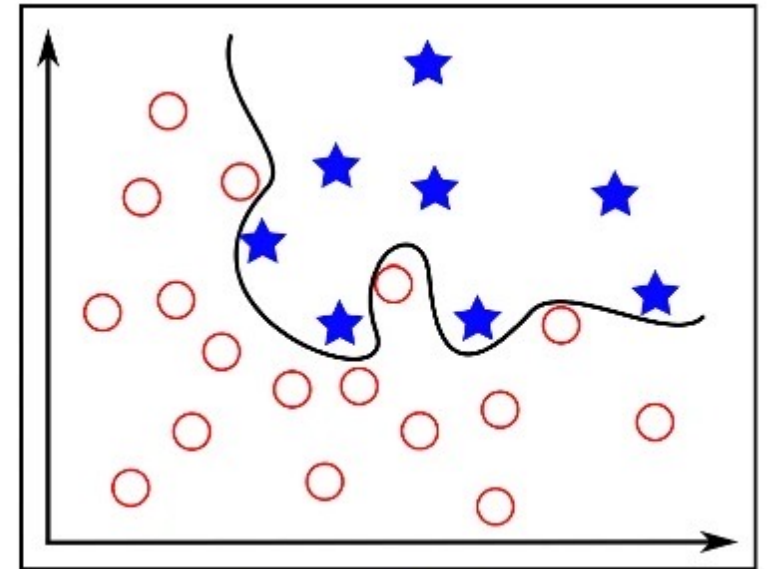
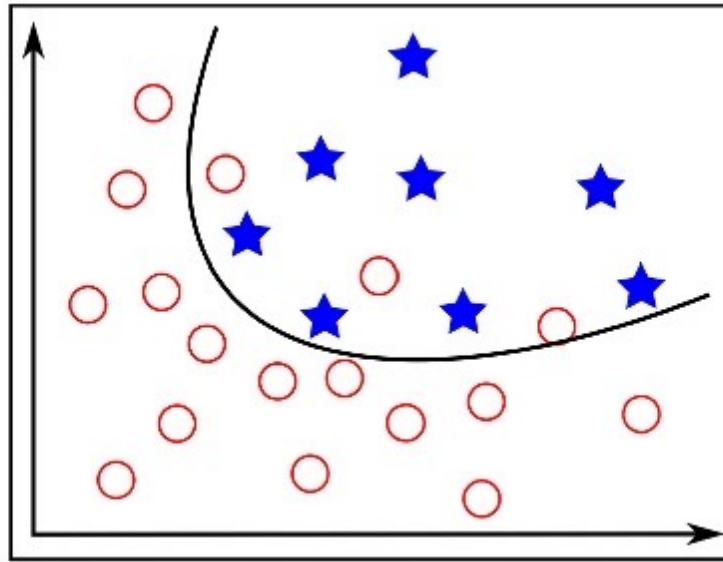
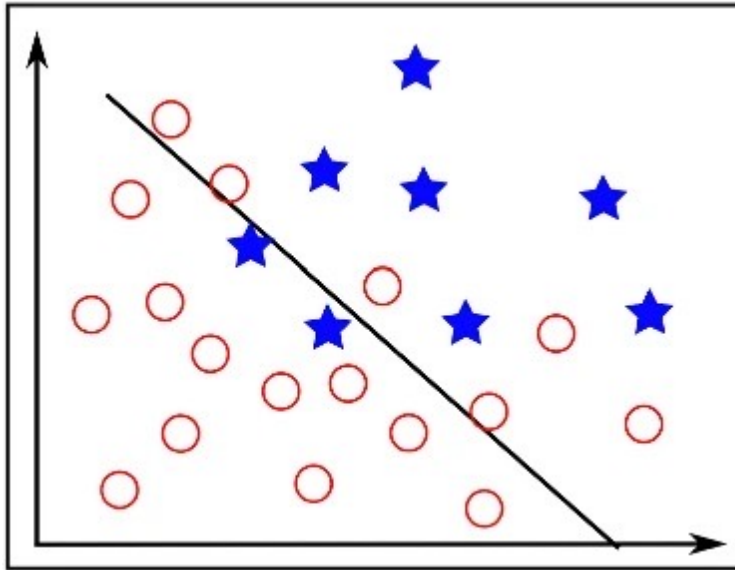


Low Approximation Error
Low Estimation Error



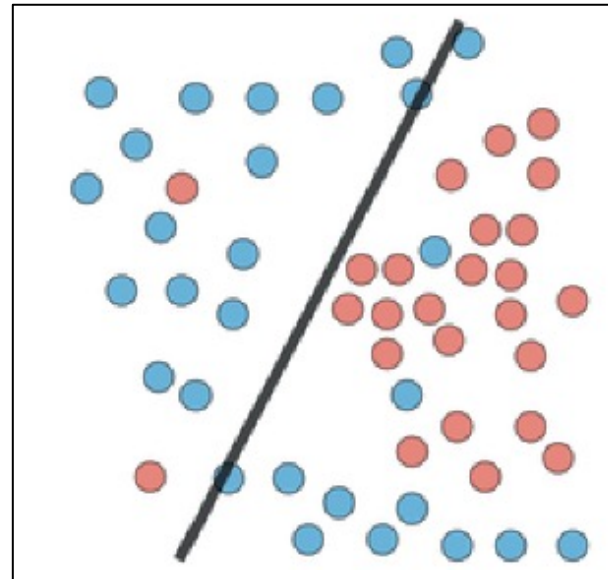
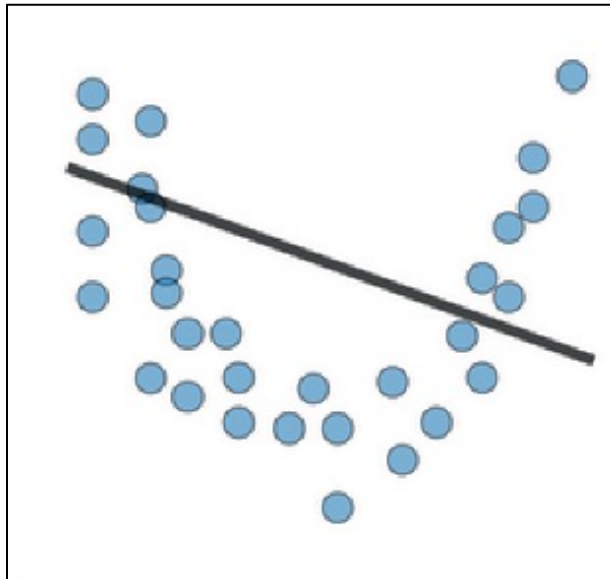
Low Approximation Error
High Estimation Error

Bias-variance trade-off: Classification



Sources of Bias & Bias Reduction

- Model class too small: unable represent underlying relationship
- Models are “too global” (e.g., constant output; single linear separator)



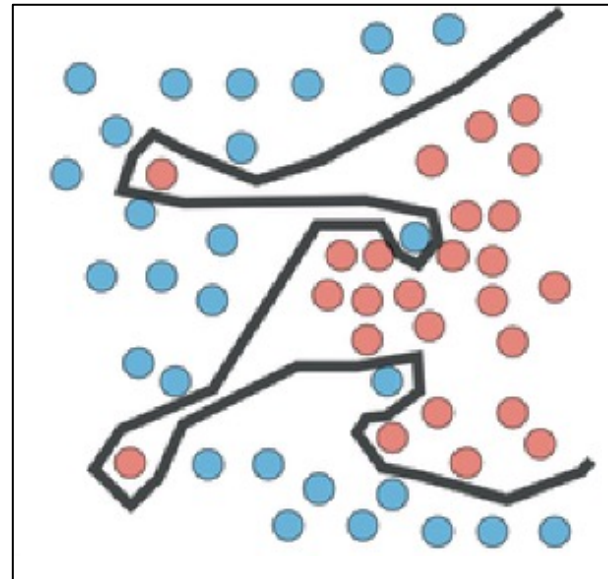
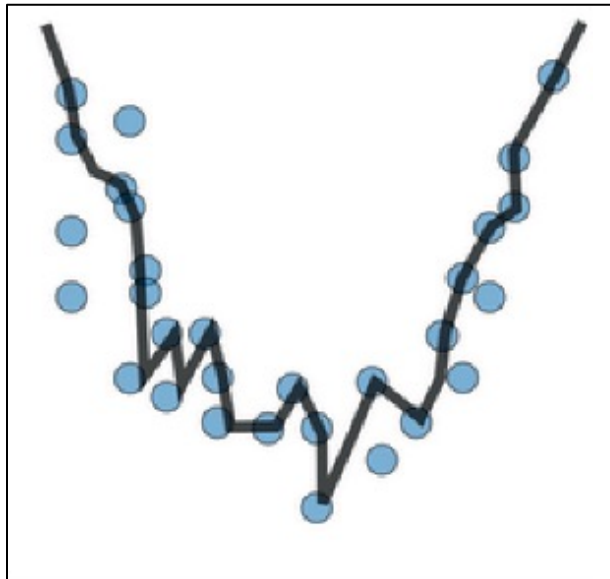
How to reduce bias?

Use more complex models

- Interaction terms
- Polynomial features
- Collect more features
- Kernels (Lec #11)
- Algorithm-specific approaches (later in the course)

Sources of Variance & Variance Reduction

- Noise in labels or features
- Models are too “local” – sensitive to small changes in feature values
- Training dataset too small



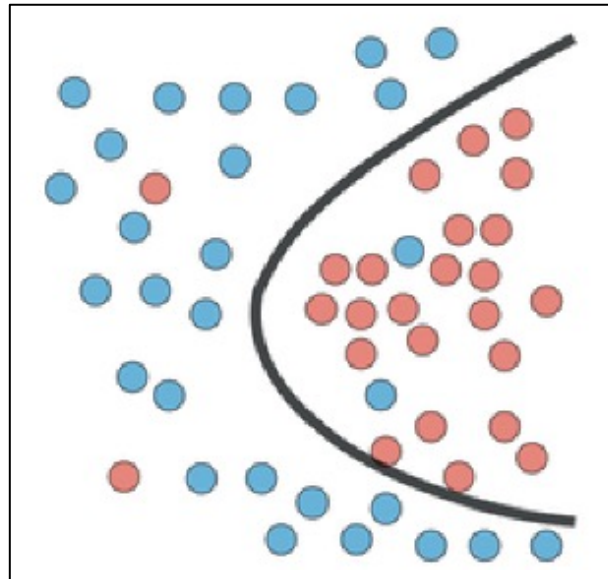
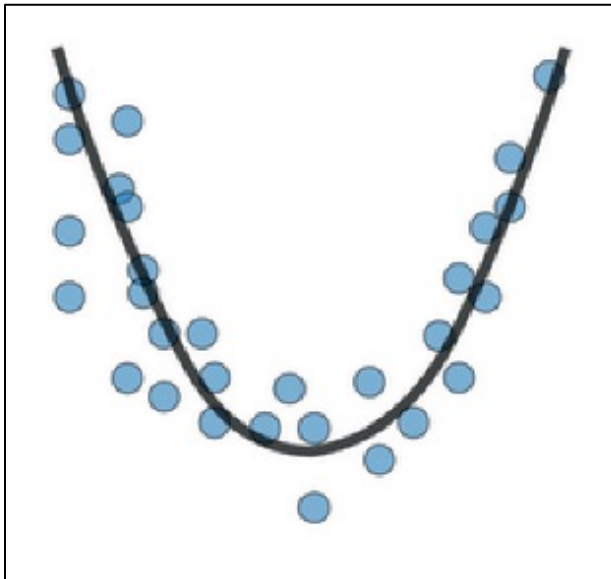
How to reduce variance?

Collect more data, or
Less complex models

- Drop interaction terms
- Regularization
- Feature selection (Lec #11)
- Don't use kernels (Lec #11)
- Ensemble (Lec #14)

Balancing Bias and Variance

- Generalizable
- not underfitting, not overfitting



Recap: Regularization

$$J(\vec{\theta}) = R_N(\vec{\theta}) + \lambda \Omega(\vec{\theta})$$

regularized empirical risk

empirical risk /
proxy for training error

regularization term/penalty

λ is the “knob” that controls how much I care about:
(1) fitting the data vs (2) reducing model complexity

If I make λ larger...

I’m paying more attention to...

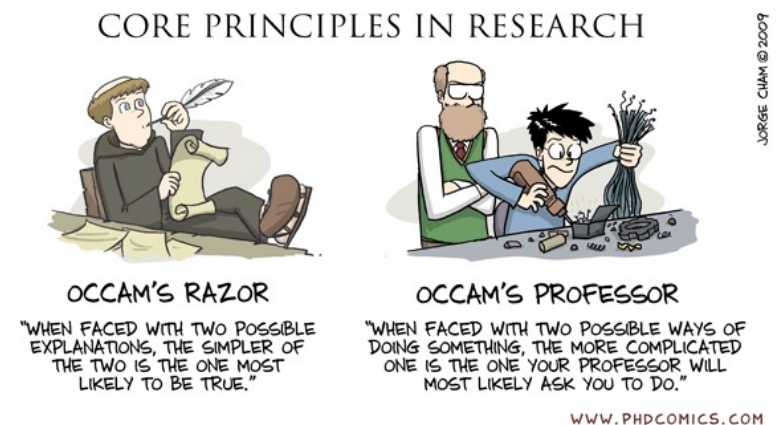
Recap: Regularization

Objective: minimize regularized empirical risk

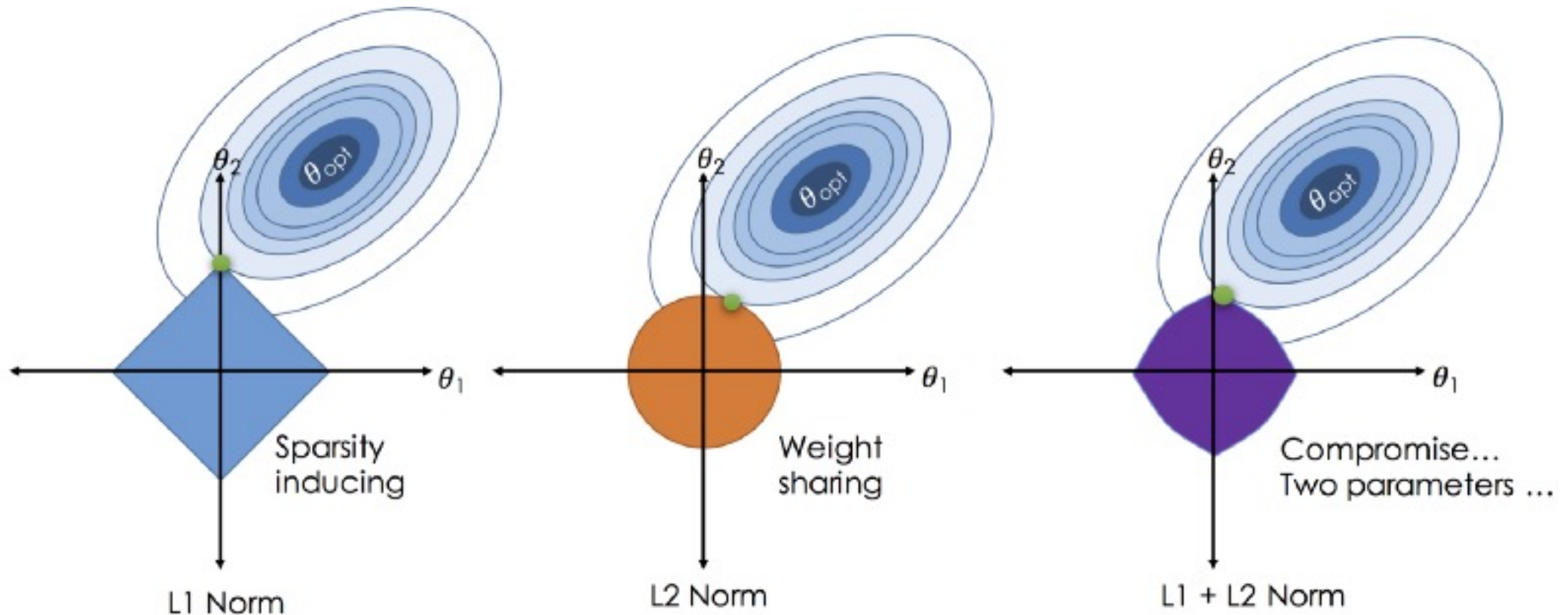
$$\min_{\vec{\theta}} R_N(\vec{\theta}) + \lambda \Omega(\vec{\theta}) \iff \min_{\vec{\theta}} R_N(\vec{\theta}) \text{ s.t. } \Omega(\vec{\theta}) \leq B$$

Intuition:

- Push model parameters to a default value of 0
- Resist setting model parameters from default value unless data strongly suggest otherwise
- Occam's Razor



Common Regularizers



Regularized Regression - Comparison

$$\|\vec{\theta}\|_2^2 = \sum_{j=1}^d \theta_j^2$$

L2-penalty encourages **small** coefficients (but not necessarily 0)
Also known as shrinkage or weight decay
Closed-form solution exists

$$\|\vec{\theta}\|_1 = \sum_{j=1}^d |\theta_j|$$

L1-penalty drives some coefficients to **zero**
Encourage **sparsity**, effectively selecting features to use
No closed-form, but efficient algorithm exists (some variants of SGD)

$$\lambda_2 \|\vec{\theta}\|_2^2 + \lambda_1 \|\vec{\theta}\|_1$$

Elastic net **balances** between ridge and LASSO
Selects features like LASSO
Shrinks coefficients of correlated predictors like ridge

Regularization: Practical Notes

- If there's an offset/intercept, this coefficient is usually left unpenalized (depending on the implementation)

$$\min_{\vec{\theta}, b} \sum_{i=1}^N \frac{(\vec{\theta} \cdot \vec{x}^{(i)} + b - y^{(i)})^2}{2} + \lambda \frac{\|\vec{\theta}\|^2}{2}$$

- Regularization strength can be “unfair” if features are on different scales

For example: x_1 = age in *years*, x_2 = weight in *grams*

- Feature preprocessing (centering + normalization) is important -> HW3

Linear Regression: sklearn implementation

<code>sklearn.linear_model.LinearRegression</code>	wrapper of <code>scipy.linalg.lstsq</code> , uses SVD
<code>sklearn.linear_model.Ridge</code>	can select solver (SGD variants, SVD, etc)
<code>sklearn.linear_model.Lasso</code>	} Lasso is just ElasticNet with L2-penalty “turned off” Implemented as coordinate descent
<code>sklearn.linear_model.ElasticNet</code>	
<code>sklearn.linear_model.SGDRegressor</code>	Generic SGD routine, mix and match different loss and regularizer

Linear Classification Model Implementation

<code>sklearn.linear_model.LogisticRegression</code>	L1/L2/elasticnet regularized logistic regression, $C = 1/\lambda$ is inverse of regularization strength
<code>sklearn.linear_model.SGDClassifier</code>	Mix and match loss and regularizer
<code>sklearn.linear_model.Perceptron</code>	SGD with perceptron loss and no regularization
<code>sklearn.svm.LinearSVC</code>	} Support Vector Machines - uses hinge loss SVC supports kernels
<code>sklearn.svm.SVC</code>	

Training sklearn models

```
from sklearn.linear_model import LinearRegression
```

```
clf = LinearRegression()  
clf.fit(X_train, y_train)
```

```
y_pred = clf.predict(X_test)
```

$$\vec{\theta}^*, b^* = \arg \min_{\vec{\theta}, b} R_N(\vec{\theta}) + \lambda \Omega(\vec{\theta})$$

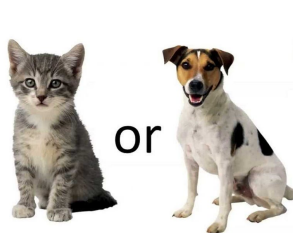
$$h(\vec{x}; \vec{\theta}, b) = \sigma(\vec{\theta} \cdot \vec{x} + b)$$

Linear Models

Classification: binary decisions

$$y \in \{-1, +1\}$$

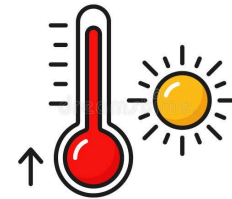
approve/decline mortgage?
cat or dog?
will it rain tomorrow?



Regression: predict values

$$y \in \mathbb{R}$$

price of house
value of stock tomorrow
highest temperature tomorrow



Some applications are not properly capture with either setup.

Linear Models

Classification: binary decisions

$$y \in \{-1, +1\}$$



- Probability of heart attack given medical conditions
- Probability of defaulting on a loan
- Probability of recidivism/reoffending

Regression: predict values

$$y \in \mathbb{R}$$

$$y \in \{-1, +1\} \text{ but want to predict } \Pr[y = +1 \mid \vec{x}] \in [0, 1]$$

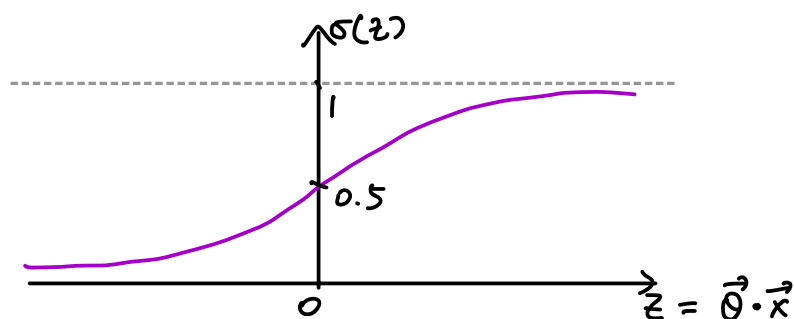
Lecture 8. Logistic Regression

Motivation: given binary labels $y \in \{-1, +1\}$
want to predict probability of positive class. $\hat{y} \in [0, 1]$

How to make a linear model output a probability? We already have $\vec{\theta} \cdot \vec{x} \in \mathbb{R}$.

→ Apply a squashing function. $\sigma: \mathbb{R} \rightarrow [0, 1]$

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{Sigmoid function.}$$



► Range of sigmoid:

→ for $z \rightarrow \infty$, $\sigma(z) \rightarrow 1$

→ for $z \rightarrow -\infty$, $\sigma(z) \rightarrow 0$

→ if $z = 0$, $\sigma(z) = 0.5$.

► Useful properties:

$$\sigma(-z) = 1 - \sigma(z)$$

$$\frac{d}{dz} \sigma(z) = \sigma(z) (1 - \sigma(z))$$

✓ continuous

✓ differentiable.

* the sigmoid function is not convex.

Logistic Regression

$$h(\vec{x}; \vec{\theta}) = \sigma(\vec{\theta} \cdot \vec{x}) = \frac{1}{1 + e^{-\vec{\theta} \cdot \vec{x}}}$$

How to train this classifier? What loss function should we use?

→ What we want: $\sigma(\vec{\theta} \cdot \vec{x})$ should be the probability of the positive class.

$$\Pr[y = +1 \mid \vec{x}] = \sigma(\vec{\theta} \cdot \vec{x})$$

Idea: if $\sigma(\vec{\theta} \cdot \vec{x})$ is truly the probability, then we can use it to write down the "likelihood" of the training data $\{\vec{x}^{(i)}, y^{(i)}\}_{i=1}^N$

► for each example $\vec{x}^{(i)}$, the likelihood/probability of seeing its label being $y^{(i)}$ is:

$$\Pr[y = y^{(i)} \mid \vec{x}^{(i)}; \vec{\theta}] = \begin{cases} \sigma(\vec{\theta} \cdot \vec{x}^{(i)}) & \text{if } y^{(i)} = +1 \\ 1 - \sigma(\vec{\theta} \cdot \vec{x}^{(i)}) & \text{if } y^{(i)} = -1 \end{cases} \Rightarrow \sigma(y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)})$$

$\underbrace{1 - \sigma(\vec{\theta} \cdot \vec{x}^{(i)})}_{= \sigma(-\vec{\theta} \cdot \vec{x}^{(i)})}$

► Since each training example is generated independently,

$$\Pr[\{y^{(i)}\}_{i=1}^N \mid \{\vec{x}^{(i)}\}_{i=1}^N; \vec{\theta}] = \prod_{i=1}^N \frac{1}{1 + e^{-y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}}}$$

Goal: maximize likelihood of training data.

$$\vec{\theta}^* = \operatorname{argmax}_{\vec{\theta}} \prod_{i=1}^N \frac{1}{1 + e^{-y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}}}$$

we normally minimize not maximize

the product looks annoying.

Problems: we want something in the form of

$$\vec{\theta}^* = \operatorname{argmin}_{\vec{\theta}} \sum_{i=1}^N \text{loss}(\vec{x}^{(i)}, y^{(i)}, \vec{\theta})$$

Tricks

product sum

① take the log. $\prod \rightarrow \sum$.

Since $\log(x)$ is increasing for $x > 0$,

$\max f(x)$ is equiv. to $\max \log(f(x))$

$$\operatorname{argmax}_{\vec{\theta}} \log \prod_{i=1}^N \frac{1}{1 + e^{-y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}}} = \operatorname{argmax}_{\vec{\theta}} \sum_{i=1}^N \log \left(\frac{1}{1 + e^{-y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}}} \right)$$

② then take the negative. $\operatorname{argmax}_x(f(x)) = \operatorname{argmin}_x(-f(x))$

$$\dots = \operatorname{argmin}_{\vec{\theta}} \sum_{i=1}^N \left(-\log \left(\frac{1}{1 + e^{-y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}}} \right) \right) = \operatorname{argmin}_{\vec{\theta}} \sum_{i=1}^N \log(1 + e^{-y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}})$$

→ Now, we can view this as an "empirical risk":

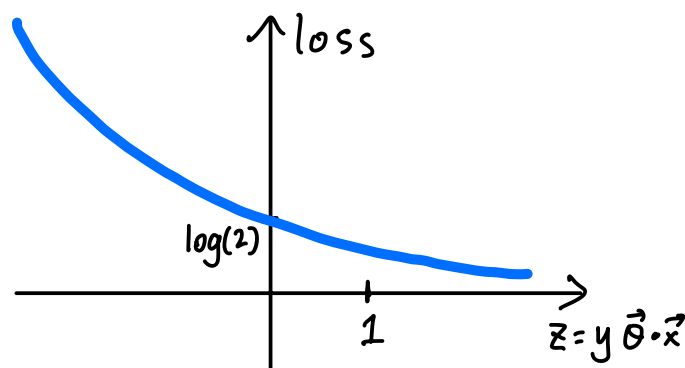
$$\text{loss}_{\log}(\vec{x}^{(i)}, y^{(i)}, \vec{\theta}) = \log(1 + e^{-y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}})$$

logistic loss:

✓ continuous

✓ differentiable

✓ convex



Let's derive the SGD update rule!

$$\begin{aligned} & \nabla_{\vec{\theta}} \log(1 + e^{-y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}}) \\ &= \frac{1}{1 + e^{-y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}}} \nabla_{\vec{\theta}} (1 + e^{-y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}}) \\ &= \frac{1}{1 + e^{-y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}}} (e^{-y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}}) \nabla_{\vec{\theta}} (-y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}) \\ &= \frac{1}{e^{y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}} + 1} (-y^{(i)} \vec{x}^{(i)}) \\ &= -y^{(i)} \vec{x}^{(i)} \sigma(-y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}) = -y^{(i)} \vec{x}^{(i)} (1 - \sigma(y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)})) \end{aligned}$$

$$\frac{d}{dx} \log(x) = \frac{1}{x}$$

$$\frac{d}{dx} e^x = e^x$$

Therefore, SGD update rule is

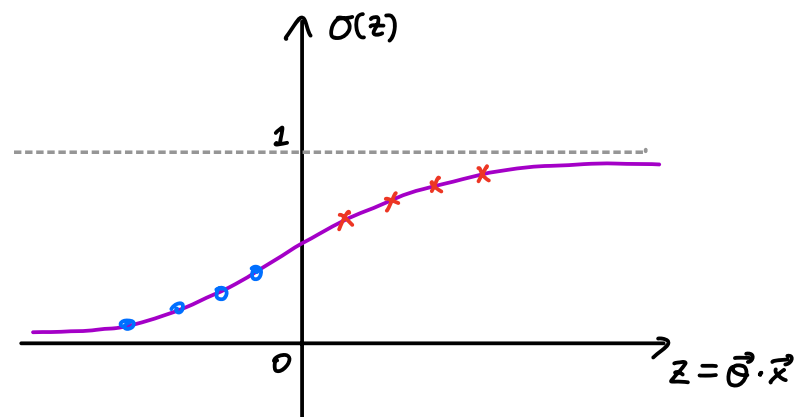
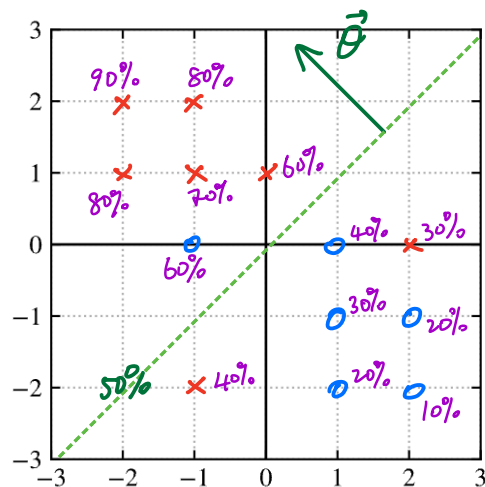
$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} - \eta_k [-y^{(i)} \vec{x}^{(i)} (1 - \sigma(y^{(i)} \vec{\theta}^{(k)} \cdot \vec{x}^{(i)}))]$$

- Unfortunately, there is no known closed-form solution for the general case.
- Since the empirical risk function is strictly convex*, there is a unique global minimum.

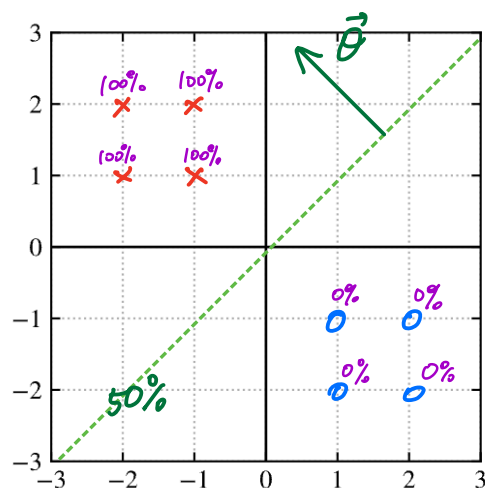
*1. unless there are linearly dependent features, in which case there are infinite number of minima that are equally good.

*2. Another pathological case:

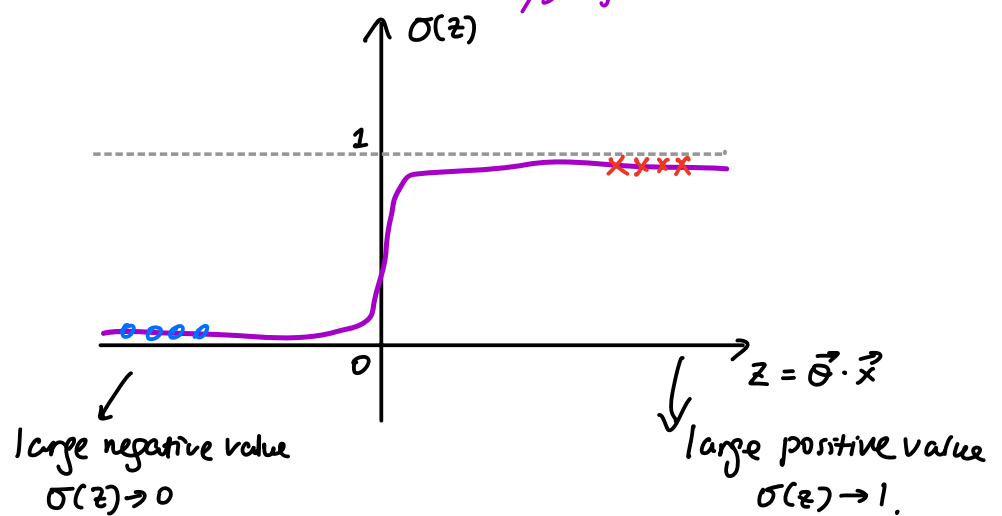
What we want logistic regression to output.



However, if data is linearly separable, and we maximize likelihood



logistic regression predicts 100% for +ve
0% for -ve



$\Rightarrow$ In-Class Exercise

How can this happen? when $\|\vec{\theta}\| \rightarrow \infty$.

Solution? Regularization.